



CIPHERWING

100% Client-Side File Cryptographic Suite

Project Report

Submitted by

GOKUL S

Reg. No. 255214344

II – M.Sc. Computer Science – “C”

BISHOP HEBER COLLEGE

Department of Computer Science

Live Deployment: <https://cipherwing.vercel.app>

Abstract

Most consumer file-encryption tools that operate through a web browser rely on a server: the file is uploaded, encrypted or decrypted remotely, and the result is sent back. This pattern, however convenient, means a third party necessarily handles the user's plaintext data at some point in the pipeline — even if only transiently — which is unacceptable for genuinely sensitive material such as financial records, legal documents, medical files, or private media.

CipherWing is a single-page web application that removes the server from this pipeline entirely. It performs authenticated AES-256-GCM encryption and decryption completely on-device, inside the user's own browser, using the native Web Crypto API. Encryption keys are derived from a user-supplied password via PBKDF2-HMAC-SHA256 with 600,000 iterations, in line with current OWASP guidance. Files are processed in 16 MB chunks inside a dedicated Web Worker so that even multi-gigabyte files can be encrypted or decrypted without freezing the browser tab or exceeding available memory. A custom binary container format, .KRYPT, stores an authenticated envelope per file, and a fully local, offline-only profile system (built on IndexedDB) allows multiple users to keep independent, PBKDF2-protected vaults on the same machine — with no backend server, no account database, and no telemetry of any kind.

This report documents the problem CipherWing addresses, surveys existing client-side and hybrid encryption tools for comparison, and details the system design, cryptographic methodology, feature set, and known limitations of the implemented solution.

1. Introduction

File encryption has traditionally required either a locally installed desktop application (VeraCrypt, 7-Zip, GnuPG) or a cloud-connected service that performs encryption on a remote server before or after transfer. The rise of powerful, standardized browser APIs — specifically the Web Crypto API, Web Workers, IndexedDB, and the File System Access API — has made a third category newly practical: an encryption tool that runs entirely inside a web page, with no installation and no server-side data handling whatsoever.

CipherWing was built to explore and demonstrate this category directly: a zero-installation, zero-knowledge file vault that a user can open in any modern browser tab, encrypt or decrypt a file of essentially any size, and close again — leaving no trace on any server anywhere, because no server ever saw the file or the password in the first place.

2. Problem Statement

- **Server-side exposure:** Most “secure” web-based file tools upload the file to a backend before or during encryption. Even reputable providers represent a single point of trust and a single point of failure — a breach, subpoena, or misconfiguration on their end exposes user data directly.
- **Installation friction:** Strong, audited desktop tools (VeraCrypt, Cryptomator) require installing native software, which is not always possible on shared, managed, or locked-down computers — exactly the situations where a quick, trustworthy encryption option is most needed.
- **Weak or legacy cipher usage:** Many lightweight browser-based or legacy desktop tools still rely on outdated primitives (e.g. unauthenticated ZipCrypto, DES, or unsalted hashing) that are cryptographically broken by modern standards and vulnerable to widely available cracking tools.
- **Memory-bound file size limits:** Naive in-browser implementations attempt to load an entire file into a single in-memory buffer before processing, which crashes the tab or the browser once file size approaches available RAM — typically well under 1 GB in practice.
- **No offline, multi-user local workflow:** Existing browser tools rarely offer a local, offline profile system, so there is no clean way for multiple people sharing one machine to keep separate encryption histories and settings without a server-backed account system.

3. Survey of Existing Systems

To position CipherWing accurately, the table below compares it against the two most widely referenced categories of client-side encryption tool: desktop volume/container encryption (VeraCrypt) and desktop/mobile cloud-file encryption (Cryptomator), alongside the general pattern of server-processed “secure” web upload tools.

Criterion	VeraCrypt	Cryptomator	Typical Web-Upload Tool	CipherWing
Installation	Native app required	Native app / mobile app required	None — browser only	None — browser only
Where encryption happens	Locally, on-device	Locally, on-device	Remote server	Locally, in-browser (Web Crypto API)
Network dependency	None	None (syncs via existing cloud client)	Required — uploads file	None — zero network calls for file data
Encryption unit	Whole volume / container	Per-file	Per-file (server-side)	Per-file, chunked (16 MB) streaming
Cipher	AES / Twofish / Serpent	AES-256-GCM (per file)	Varies — often unaudited	AES-256-GCM, authenticated per chunk
Multi-user on one device	Manual, no built-in accounts	Manual, no built-in accounts	Server-side account system	Local offline profiles (IndexedDB + PBKDF2)
Large file handling	Native OS I/O, no practical limit	Native OS I/O, no practical limit	Limited by upload size / server memory	Streamed 16 MB chunks, 2 GB+ tested in-browser

Table 1: Comparative positioning of CipherWing against established client-side and web-upload encryption tools, based on each project's published documentation and independent 2026 reviews.

VeraCrypt remains the benchmark for full-volume, on-the-fly encryption and is independently audited, but it is a native desktop application, encrypts by container rather than by individual file (making incremental cloud sync of a large container impractical), and has a steeper setup curve for non-technical users. Cryptomator solves the per-file cloud-sync problem elegantly and is also independently audited (Cure53,

OSTIF/QuarksLab), but likewise requires a native or mobile client to be installed and is designed around syncing through an existing cloud provider rather than standalone offline use. Neither tool is usable directly from an unmodified web browser.

General-purpose “secure” web upload tools solve the installation problem but reintroduce the exact exposure CipherWing is designed to eliminate: the plaintext file (or, in weaker implementations, the password itself) transits a network and is handled, however briefly, by a server the user does not control.

4. Proposed System — CipherWing

CipherWing is proposed as a fourth category: a zero-installation, zero-knowledge, browser-native file vault. It combines the “no server touches your data” guarantee of VeraCrypt/Cryptomator with the “no installation required” convenience of a web upload tool, without inheriting either one's drawback.

4.1 Design Objectives

- Perform authenticated, industry-standard encryption (AES-256-GCM) entirely within the browser sandbox, with zero file or password bytes ever leaving the device.
- Support files of at least 2 GB without exceeding practical browser memory limits, by processing data in fixed-size streamed chunks.
- Keep the UI thread responsive throughout processing by running all cryptographic work in a dedicated Web Worker.
- Provide a local, offline, multi-profile authentication layer so that more than one person can use the same installation-free deployment on a shared computer without a backend account system.
- Fail safely: any tampering, truncation, or wrong password must halt processing and report an error rather than silently returning corrupted output.
- Remain usable directly from a static, serverless deployment (Vercel), with no backend runtime of any kind.

5. System Architecture

The application is structured as a static single-page site with no backend runtime. All modules run client-side, split across the browser's main (UI) thread and a dedicated background Web Worker, coordinated entirely through message passing and the browser's own storage and cryptography primitives.

Layer / Module	File	Responsibility
UI Controller	app.js	DOM state, drag-and-drop queue, theme + language toggles, spawns the Worker, streams output via the File System Access API where supported.
Cryptographic Engine	worker.js	Runs off the main thread. Key derivation (PBKDF2), chunked AES-256-GCM encryption/decryption, .KRYPT container construction/parsing.
Local Authentication	accounts.js	Creates and verifies local offline profiles; PBKDF2-hashes profile passwords; constant-time hash comparison.
Local Database	db.js	Promise-based wrapper around IndexedDB; manages the profiles and history object stores.
Audit Log	history.js	Renders a searchable, filterable local log of encrypt/decrypt operations; tracks browser storage quota usage.
3D Visual Layer	screen.js	Optional Three.js WebGL background; purely presentational, no security-relevant logic.
Batch Export	jszip_min.js	Third-party library used to bundle multiple processed files into a single downloadable ZIP archive.

Table 2: Module responsibilities within the CipherWing client-side architecture.

Because no module requires server-side execution, the entire application is hosted as static assets (see `vercel.json`) with hardened HTTP response headers — including `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, and `Referrer-Policy: no-referrer` — applied at the CDN edge rather than by any application server.

6. Cryptographic Methodology

6.1 Key Derivation

A user's password is never used directly as an encryption key. It is passed through PBKDF2 with HMAC-SHA-256 and 600,000 iterations against a unique, randomly generated 16-byte salt, producing a 256-bit key suitable for AES-256. The 600,000-iteration figure matches the current OWASP Password Storage Cheat Sheet minimum for PBKDF2-SHA256 deployments, chosen specifically to impose a meaningful, GPU-resistant time cost on offline brute-force attempts against a stolen file.

6.2 Authenticated Encryption

Actual file content is encrypted with AES-256-GCM, an authenticated encryption mode that produces both ciphertext and a 128-bit authentication tag per operation. Every chunk receives its own derived Initialization Vector (IV), computed by offsetting a random 12-byte base IV by the chunk's index, guaranteeing that no IV is ever reused under the same key — a critical requirement, since IV reuse is one of the most common real-world failure modes of AES-GCM implementations.

6.3 Additional Authenticated Data (AAD)

Each chunk's ciphertext is authenticated not just against its own content, but against a binding of the file header, the chunk's index, and an is-last flag. This means an attacker who intercepts a .KRYPT file cannot reorder, truncate, duplicate, or splice chunks without the decryption step detecting the tampering and aborting — the file format is structurally resistant to chunk-level manipulation, not merely to content modification.

6.4 The .KRYPT Container Format

CipherWing defines its own compact binary envelope rather than reusing a general-purpose archive format. The header — magic bytes, version, flags, salt, base IV, and an optional plaintext hint — is stored unencrypted by design, so the application can display a user-supplied hint before a password is even entered. Every field after the header (the true filename, MIME type, size, and all file content) is individually authenticated-encrypted, so the container's structure is inspectable while its content remains unrecoverable without the correct key.

6.5 Local Authentication

Profile passwords for the offline, multi-user login layer use the same PBKDF2 (600,000 iterations, SHA-256) construction as file encryption, with a per-profile random salt. Login comparison uses a constant-time bitwise XOR/OR reduction over the stored and computed hashes rather than a short-circuiting equality check, specifically to prevent timing side-channel attacks that could otherwise leak information about the correct hash byte-by-byte.

7. Key Features

- AES-256-GCM authenticated encryption and decryption, fully client-side.
- PBKDF2-HMAC-SHA256 key derivation at 600,000 iterations (file encryption and local login share this standard).
- Chunked streaming pipeline (16 MB per chunk) supporting files of 2 GB and above without holding the whole file in memory.
- Dedicated Web Worker execution — the UI thread never blocks during processing.
- File System Access API integration for direct-to-disk streaming output on supported browsers (Chrome, Edge, Opera).
- Custom .KRYPT binary container with an inspectable, unencrypted hint field and fully authenticated content.
- Optional Gzip compression (via CompressionStream) prior to encryption.
- Batch processing with ZIP export (via JSZip) for multiple files at once.
- Local, offline, multi-profile authentication system backed by IndexedDB — no server account database.
- Searchable, filterable local processing history with live browser storage-quota reporting.
- Password-strength meter and a secure password generator.
- Live light/dark theming and full English ↔ Tamil bilingual UI, switchable without reload.
- Interactive Three.js WebGL padlock background with mouse parallax and a success-state animation.
- beforeunload tab-close protection during active file processing to prevent accidental data loss.

8. Technology Stack

Category	Technology
Cryptography	Web Crypto API — SubtleCrypto (PBKDF2, AES-256-GCM)
Concurrency	Web Workers (worker.js runs off the main UI thread)
Storage	IndexedDB (profiles, history) · sessionStorage (login session)
Large-file I/O	File System Access API (showSaveFilePicker, streamed writes)
Compression	CompressionStream (native Gzip)
Batch export	JSZip (bundled third-party library)
3D graphics	Three.js r128 (WebGL, loaded from CDN)
Frontend	Vanilla JavaScript (no framework), HTML5, CSS3 (custom design system, CSS variables)
Hosting	Vercel — static hosting only, no backend runtime, hardened response headers

9. Advantages Over Existing Systems

- **No installation, no account server:** Unlike VeraCrypt or Cryptomator, CipherWing requires nothing beyond a modern browser tab — and unlike a typical web upload tool, it still never sends data to a server.
- **Authenticated encryption throughout:** Every chunk, not just the file as a whole, carries its own authentication tag and index-bound AAD, giving CipherWing tamper and reordering detection that many simpler browser tools do not implement.
- **Practical large-file support in the browser:** The 16 MB chunked pipeline lets CipherWing handle multi-gigabyte files directly in a browser tab, a size class that naive single-buffer browser implementations cannot reach without crashing.
- **Transparent, honest security messaging:** The application explicitly documents what its local-profile passwords do and do not protect against (shared-device convenience, not DevTools-level storage inspection), avoiding the overstated security claims common in this space.

10. Limitations

- **Password is the sole root of trust:** As with any zero-knowledge design, a forgotten password makes the file permanently unrecoverable — there is no backdoor or reset mechanism, by design.
- **File System Access API browser support is uneven:** Direct-to-disk streaming (showSaveFilePicker) is available in Chrome, Edge, and Opera on desktop, but Firefox and Safari currently expose only the sandboxed Origin Private File System and do not support the local-disk save picker — on those browsers CipherWing falls back to accumulating output in memory, which is impractical for very large files.
- **No protection against a compromised or physically accessed device:** Because everything happens client-side, CipherWing cannot defend against malware, browser extensions, or a user with DevTools access to an already-unlocked session — its threat model is network/server exposure, not endpoint compromise.
- **PBKDF2, not a memory-hard function:** Current OWASP guidance now lists Argon2id as the preferred general-purpose password-hashing algorithm, with PBKDF2 retained primarily for FIPS-140 compliance contexts; PBKDF2 remains resistant to brute-forcing at 600,000 iterations but is

more GPU-parallelizable than a memory-hard alternative (see Section 12 and the accompanying roadmap document for a proposed migration path).

11. Conclusion

CipherWing demonstrates that strong, authenticated, large-file encryption does not require trusting a third-party server, nor does it require installing native software. Modern browser primitives — the Web Crypto API, Web Workers, IndexedDB, and the File System Access API — are sufficient to build a private, offline-capable, multi-user file vault that runs entirely inside a browser tab. By pairing a custom authenticated container format (.KRYPT) with a chunked streaming pipeline, the project shows that security and practical usability can scale together, even to multi-gigabyte files, on ordinary consumer hardware.

12. Future Enhancements

A detailed, phased implementation roadmap — covering an Argon2id migration path, WebAuthn/passkey authentication, a WASM-accelerated cipher core, peer-to-peer encrypted transfer, installable PWA support, and associated security hardening — is maintained as a separate companion document (CipherWing_Feature_Security_Roadmap.md) so that it can be tracked and executed independently of this report.